Git Advanced Commands

A practical guide with meanings, examples, and when to use each command.

Topics covered

- Stash and restore changes
- Rebase and cherry-pick commits
- Bisect to locate bugs
- Reset, revert, and amend commits
- Squash commits
- Tags and commit history
- Diff, clean, fetch, pull, and force push
- Reflog for recovery

Practise in a test repository first. Commands that rewrite history or delete files need extra care.

1. Git Stash

Command:

```
git stash
git stash pop
git stash list
git stash apply
```

Meaning: Temporarily stores tracked-file changes so you can work on something else without committing them. `pop` restores the latest stash and removes it from the stash list; `apply` restores it but keeps the stash.

When to use: When you need to switch branches or handle urgent work while your current changes are unfinished.

Example:

```
git stash
git switch main
# do the other task
git switch feature
git stash pop
```

Tip: `git stash -u` also includes untracked files.

2. Git Rebase

Command:

```
git rebase main
```

Meaning: Replays the commits from your current branch on top of the latest commit in `main`, creating a more linear history.

When to use: When you want to bring your feature branch up to date with main.

Example:

```
git switch feature
git fetch origin
git rebase origin/main
```

If conflicts occur, resolve them, run `git add `, then `git rebase --continue`. Use `git rebase --abort` to cancel. Avoid rebasing shared branches unless your team agrees.

3. Git Cherry-Pick

Command:

```
git cherry-pick <commit_id>
```

Meaning: Applies the changes introduced by one selected commit onto your current branch, creating a new commit.

When to use: When you need one particular fix or feature from another branch without merging the entire branch.

Example:

```
git switch main
git cherry-pick a1b2c3d
```

If conflicts occur, resolve them and run `git cherry-pick --continue`, or use `git cherry-pick --abort`.

4. Git Bisect

Command:

```
git bisect start
git bisect bad
git bisect good <known_good_commit>
# test each version and mark it:
git bisect good
git bisect bad
git bisect reset
```

Meaning: Uses a binary search through commit history to identify the commit that introduced a bug.

When to use: When you know a version that worked and a later version that fails, but do not know which change caused the problem.

Example:

```
git bisect start
git bisect bad
git bisect good v1.0
# test the checked-out commit, then mark it good or bad
git bisect reset
```

Mark each checked commit accurately. `git bisect reset` returns you to the original branch position.

5. Git Reset

Command:

```
git reset --soft HEAD~1
git reset --mixed HEAD~1
git reset --hard HEAD~1
```

Meaning: Moves the current branch pointer to an earlier commit. The options control what happens to the changes.

When to use: When you need to undo a local commit or unstage changes.

Example:

```
git reset --soft HEAD~1 # keep changes staged
git reset --mixed HEAD~1 # keep changes unstaged
git reset --hard HEAD~1 # discard tracked changes
```

Warning: `--hard` can permanently discard uncommitted tracked-file changes. Resetting commits already shared with others can disrupt their work.

6. Git Revert

Command:

```
git revert <commit_id>
```

Meaning: Creates a new commit that reverses the changes introduced by an earlier commit.

When to use: When you need to undo a commit that has already been pushed or shared, without rewriting history.

Example:

```
git log --oneline
git revert a1b2c3d
git push origin main
```

For a merge commit, reverting requires choosing a mainline parent with `-m`; check carefully before doing so.

7. Git Amend

Command:

```
git commit --amend
git commit --amend -m "Updated message"
```

Meaning: Replaces the most recent commit with a new commit, allowing you to change its message or include newly staged changes.

When to use: When you forgot a file or made a small correction to your latest local commit.

Example:

```
git add forgotten_file.txt
git commit --amend
```

Amending changes the commit ID. Avoid amending a commit that has already been pushed unless you understand the consequences.

8. Git Squash

Command:

```
git rebase -i HEAD~3
```

Meaning: Combines multiple commits into fewer commits, often one clean commit. In the interactive editor, keep the first commit as `pick` and mark later commits as `squash` or `s`.

When to use: When several small development commits should become one meaningful commit before sharing or merging.

Example:

```
git rebase -i HEAD~3
# In the editor, for example:
# pick <commit-1> First change
# squash <commit-2> Small fix
# squash <commit-3> Final update
```

Save and close the editor, then edit the combined commit message if prompted. This rewrites history.

9. Git Tag

Command:

```
git tag v1.0
git tag
git push origin v1.0
git push origin --tags
```

Meaning: Adds a named marker to a specific commit, commonly used to identify software releases.

When to use: When marking versions such as v1.0 or v2.0.

Example:

```
git tag -a v1.0 -m "First release"
git push origin v1.0
```

Annotated tags (`-a`) store a message and metadata and are commonly preferred for releases.

10. Git Log

Command:

```
git log
git log --oneline
git log --oneline --graph --all
```

Meaning: Displays commit history. The graph option shows branch and merge structure.

When to use: When you want to see who committed changes, when they were made, or how branches developed.

Example:

```
git log --oneline --graph --all
```

11. Git Diff

Command:

```
git diff
git diff --staged
git diff HEAD
git diff branch1 branch2
```

Meaning: Shows differences between working files, staged changes, commits, or branches.

When to use: Before committing, to review exactly what has changed.

Example:

```
git diff # unstaged changes
git diff --staged # staged changes
git diff HEAD # all changes since last commit
```

12. Git Clean

Command:

```
git clean -n
git clean -f
git clean -fd
```

Meaning: Removes untracked files and, with `-d`, untracked directories. `-n` is a dry run that only previews what would be removed.

When to use: When you want to remove generated or unnecessary untracked files from the working tree.

Example:

```
git clean -n # preview first
git clean -fd # remove untracked files and directories
```

Warning: this deletes untracked files outside Git's normal commit history. Always run the dry-run command first.

13. Git Fetch

Command:

```
git fetch origin
```

Meaning: Downloads new commits and references from the remote repository but does not merge them into your current branch.

When to use: When you want to inspect remote updates before integrating them.

Example:

```
git fetch origin
git log --oneline HEAD..origin/main
```

14. Git Pull

Command:

```
git pull
```

Meaning: Fetches remote changes and integrates them into the current branch. Depending on configuration, integration may use merge or rebase.

When to use: When you want to update your local branch with changes from its configured remote branch.

Example:

```
git switch main
git pull origin main
```

To make the strategy explicit, use `git pull --rebase` or `git pull --no-rebase`.

15. Git Push Force (with lease)

Command:

```
git push --force-with-lease origin <branch>
```

Meaning: Updates a remote branch even when the update is not a fast-forward, but checks that the remote branch has not changed unexpectedly since your last fetch.

When to use: Only when you intentionally rewrote your own branch history, for example after an interactive rebase.

Example:

```
git fetch origin
git push --force-with-lease origin feature
```

Warning: force-pushing can overwrite other people's work. Coordinate with your team and avoid using it on shared branches.

16. Git Reflog

Command:

```
git reflog
git reflog show --date=local
```

Meaning: Shows the recent movements of HEAD and branch references in your local repository, including many commits no longer visible in normal log history.

When to use: When you accidentally reset, rebase, or switch references and need to find a previous commit.

Example:

```
git reflog
# identify the commit you want to recover
git switch -c recovery <commit_id>
```

Reflog is local to your repository and entries eventually expire. It is a recovery aid, not a permanent backup.

Quick Reference Table

No.	Command	Purpose
1	git stash / git stash pop	Temporarily save / restore changes
2	git rebase main	Replay current branch commits on main
3	git cherry-pick <commit_id>	Apply one specific commit
4	git bisect	Find the commit that introduced a bug
5	git reset	Move branch pointer; undo local commits
6	git revert <commit_id>	Undo a commit with a new commit
7	git commit --amend	Modify the latest commit
8	git rebase -i HEAD~3	Interactively combine or edit commits
9	git tag v1.0	Mark a release commit
10	git log --oneline --graph --all	View commit history and branch graph
11	git diff	Compare changes
12	git clean -n / -f / -fd	Preview / remove untracked files
13	git fetch origin	Download remote updates only
14	git pull	Fetch and integrate remote updates
15	git push --force-with-lease	Update remote after history rewrite
16	git reflog	Inspect recent reference movements / recover commits

Practice tip: Try these commands in a separate test repository before using them on important projects. Commands such as reset --hard, clean -f, rebase, and force push need extra care.